
xv6 Process Tree Visualizer

CSCI-315 Operating Systems — Project Discussion

A pstree-like tool for xv6 that displays parent-child process relationships,
PIDs, and process states via a new kernel system call

 github.com/Darth-218/xv6-ptv

Team of 5 | 18 Commits Ahead of Upstream

AGENDA

01 Project Overview

Goals, architecture, and data flow

02 Kernel Implementation

Data structures, syscall plumbing, and handler

03 User-Space Implementation

The ptv visualizer, DFS algorithm, and output

04 Testing & Validation

Test programs, automation, and results

05 Key Learnings & Discussion

Results, insights, and future work

01

Project Overview

Understanding the problem, goals, and system architecture
of the Process Tree Visualizer for xv6

What is ptv?

ptv (Process Tree Visualizer) is a user-space program for xv6 that displays parent-child process relationships, PIDs, and states — like Linux's **ps**.

Retrieves data via new syscall **getprocs()** with state annotations:

ZOMBIE — exited, awaiting parent wait()

RUN — currently executing

SLEEP — sleeping on a channel

RDY — ready to run

EXAMPLE OUTPUT

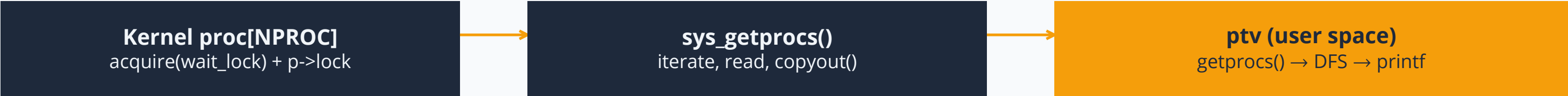
```
$ ptv
init(1,SLEEP)
└─ sh(2,SLEEP)
   └─ ptv(4,RUN)
Total: 3 processes
```



Key Innovation

Kernel-to-user data exposure through **struct pinfo** — a shared data structure that safely copies process metadata (pid, ppid, state, name) from kernel space to user space using the **copyout()** mechanism.

Architecture & Data Flow



Boundary Crossing

Layer	Address Space	Mechanism
sys_getprocs()	Kernel	Reads proc[NPROC] with wait_lock + p->lock
copyout()	Kernel → User	Copies struct pinfo to user buffer
getprocs() stub	User	ecall trampoline (usys.S)
ptv main	User	DFS tree construction + printf

Locking Protocol

Lock	Acquired	Protects	Held During
wait_lock	Once	p->parent pointer	Entire iteration
p->lock	Per-entry	p->state, pid, name	Reading single entry

02

Kernel Implementation

Shared data structures, system call plumbing,
and the syscall handler implementation

struct pinfo & Syscall Plumbing

kernel/pinfo.h

```
// State constants (PINFO_ prefix avoids
// conflict with kernel enum procstate)
#define PINFO_UNUSED 0
#define PINFO_USED 1
#define PINFO_SLEEPING 2
#define PINFO_RUNNABLE 3
#define PINFO_RUNNING 4
#define PINFO_ZOMBIE 5
struct pinfo {
    int pid; int ppid; int state;
    char name[16];
};
```

Syscall Wiring

```
// kernel/syscall.h
#define SYS_getprocs 22
// kernel/syscall.c — extern declaration
extern uint64 sys_getprocs(void);
// Dispatch table entry
static uint64 (*syscalls[])(void) = {
    ...
    [SYS_getprocs] sys_getprocs,
};
```

Design Decisions

Enum Conflict Resolution: Kernel's proc.h defines enum procstate {UNUSED, USED, SLEEPING...} which conflicts with user-space macros. Solved by prefixing: PINFO_UNUSED, PINFO_USED, etc.

Dispatch Table: Designated initializers [SYS_getprocs] automatically expand the array to index 22. NELEM(syscalls) uses static array size for bounds checking.

Forward Declaration: user/user.h uses struct pinfo; forward declaration to keep the header minimal — full definition comes from kernel/pinfo.h.

sys_getprocs() Implementation

kernel/sysproc.c

```
uint64 sys_getprocs(void) {
    uint64 buf; int max; struct pinfo pi;
    struct proc *p; int count = 0;
    argaddr(0, &buf); argint(1, &max);
    if (max <= 0) return -1;
    acquire(&wait_lock);
    for (p=proc; p<&proc[NPROC] &&
        count<max; p++) {
        acquire(&p->lock);
        if (p->state==UNUSED) {
            release(&p->lock); continue; }
        pi.pid=p->pid; pi.state=p->state;
        safestrcpy(pi.name,p->name,16);
        pi.ppid=p->parent?p->parent->pid:-1;
        release(&p->lock);
        if(copyout(myproc()->pagetable,
            buf+count*sizeof(pi),
            (char*)&pi,sizeof(pi))<0){
            release(&wait_lock);return-1;}
        count++; }
    release(&wait_lock);returncount;
}
```

Edge Cases Handled

Case	Behavior
UNUSED slots	Skipped silently
Zombie process	Included, state=5
Orphan / Init	ppid = -1
Buffer full	Partial count returned
copyout() fails	Release locks, return -1
max_count <= 0	Immediate -1
Empty table	Returns 0 (not error)

Syscall Contract

Input: a0 = user buffer addr, a1 = max_count | **Returns:** a0 = count copied (>0), 0, or -1 on error

Guarantees: Each struct pinfo has valid pid, ppid (-1 if no parent), state (0-5), name (NUL-terminated, max 16 bytes)

Locks: wait_lock (entire iteration) + p->lock (per-entry) — released in reverse order to prevent deadlock

03

User-Space Implementation

The visualizer program, syscall trampoline,
and tree rendering algorithm

The ptv Visualizer (84 lines)

user/ptv.c — Core Functions

```
// State-aware formatting with indentation
void print_process(struct pinfo*p,intd){
for(int i=0;i<d;i++)
printf(i==d-1?" |-- ":" ");
if(p->state==PINFO_ZOMBIE)
printf("%s(%d,ZOMBIE)\n",p->name,p->pid);
else if(p->state==PINFO_RUNNING)
printf("%s(%d,RUN)\n",p->name,p->pid);
else if(p->state==PINFO_SLEEPING)
printf("%s(%d,SLEEP)\n",p->name,p->pid);
else if(p->state==PINFO_RUNNABLE)
printf("%s(%d,RDY)\n",p->name,p->pid);
else printf("%s(%d)\n",p->name,p->pid);
}
// O(n^2) scan, n <= NPROC = 64
void dfs(int ppid,int d,struct pinfo*pr,
int n,int*c){
for(int i=0;i<n;i++)
if(pr[i].ppid==ppid){
(*c)++;print_process(&pr[i],d);
dfs(pr[i].pid,d+1,pr,n,c);}
}
```

Algorithm (main)

1. Call getprocs(procs, NPROC) to fetch all process data from kernel
2. Locate PID 1 (init) as the tree root
3. Print root with state annotation
4. Recursively DFS children where procs[i].ppid == parent_pid
5. Print "Total: N processes" summary

State Label Mapping

State	Value	Label	Example Output
ZOMBIE	5	ZOMBIE	zombie(3,ZOMBIE)
RUNNING	4	RUN	ptv(4,RUN)

DFS Algorithm & Output Format

Output Examples

Basic: init → sh → ptv

```
$ ptv
init(1,SLEEP)
|-- sh(2,SLEEP)
|-- ptv(4,RUN)
Total: 3 processes
```

After forktree: 4-level hierarchy

```
$ forktree
forktree starting
forktree done
$ ptv
init(1,SLEEP)
|-- sh(2,SLEEP)
|-- forktree(4)
|-- forktree(5)
|-- forktree(6,ZOMBIE)
Total: 6 processes
```

Implementation Notes

Complexity

$O(n^2)$ linear scan where $n \leq \text{NPROC} = 64$. Since the process table is small, no adjacency list is needed — a simple nested loop suffices.

Indentation

Each depth level adds 6 characters: " |-- " for the last level, " " for ancestor levels. Creates a clean hierarchical tree view.

Root Discovery

PID 1 (init) is the fixed tree root. The algorithm explicitly searches for init in the returned process array before starting DFS.

State Filtering

UNUSED processes (state=0) are filtered at the kernel level — they never reach user space. Only active processes appear in output.

04

Testing & Validation

Test programs, edge case coverage,
and automated validation

Test Programs

 forktree.c

Multi-level fork hierarchy

Creates a 4-level process tree with double-fork technique:

forktree(4) → forktree(5) → forktree(6,ZOMBIE)

Grandparent exits immediately to unblock shell.
Middle process and descendants persist for ptv inspection.

 orphantree.c

Orphan reparenting test

Child forks grandchild, then exits. Kernel's reparent() in proc.c:311 reparents grandchild to init (PID 1).

Verifies ptv shows orphaned process under init in the tree.

 zombie.c

Zombie process test

Double-fork creates a zombie while keeping shell unblocked:

- 1. Original forks child, exits
- 2. Child forks grandchild
- 3. Grandchild exits → ZOMBIE
- 4. Child stays alive (pause loop)
- 5. ptv shows zombie(N,ZOMBIE)

Test Coverage Matrix

Feature	forktree	orphantree	zombie
Tree hierarchy	4-level depth	1-level	1-level
State display	ZOMBIE (leaf)	SLEEP (pause)	ZOMBIE marker
Orphan handling	—	Reparented to init	—
Shell blocking	Double-fork	Double-fork	Double-fork

Automated Test Suite

test-xv6.py — Test Functions

```
# Boot xv6, run pty, validate output
def test_ptv_basic():
    q = QEMU(True)
    q.cmd("pty\n")
    lines = q.lines()
    assert match(r'^init\(', lines) # root
    assert search(r'sh\(', lines) # shell
    # Run forktree, verify hierarchy
    def test_ptv_forktree():
        q = QEMU(True)
        q.cmd("forktree\n"); sleep(3)
        q.cmd("pty\n")
        assert search(r'forktree\(', q.lines())
    # Orphan + zombie tests (similar pattern)
    def test_ptv_orphans(): ...
    def test_ptv_zombies(): ...
```

Test Validation

Test	Validates
test_ptv_basic	init is tree root, sh is visible child
test_ptv_forktree	Multi-level fork hierarchy visible
test_ptv_orphans	Orphan reparented to init appears
test_ptv_zombies	Zombie marked with ,ZOMBIE suffix

Running Tests

```
$ ./test-xv6.py pty_basic
$ ./test-xv6.py pty_forktree
$ ./test-xv6.py pty # all 4 tests
```


Key Learnings & Future Work

Issues Resolved

- ✓ State #define conflict with enum procstate → **PINFO_** prefix
- ✓ Regex `^init 1` didn't match `init(1,SLEEP)` → `^init\1`
- ✓ Function names used `pstree` instead of `ptv` → renamed to **test_ptv_***

Current Considerations

- `forktree.c` leaf process exits into ZOMBIE state — tree depth math verified for `max_depth=4`
- `test_ptv_forktree` relies on `time.sleep(3)` being sufficient for fork hierarchy to settle — could be flaky on slow emulation

Potential Improvements

- Command-line flags** — Add `-a` (show all processes including UNUSED) and `-p PID` (show ancestry for a single process) flags for more flexible inspection
- Cycle detection** — Add safety-net cycle detection in DFS (unlikely in xv6's tree-structured process hierarchy, but robust for future kernel modifications)
- Orphaned root display** — Print orphaned root processes separately instead of requiring PID 1 as the universal root, supporting more complex process topologies
- Performance** — Build adjacency list from `pinfo` array for $O(n)$ tree construction (currently $O(n^2)$ is acceptable since $n \leq 64$)

Questions & Discussion

Thank you for your attention

 github.com/Darth-218/xv6-ptv

CSCI-315 Operating Systems | Team of 5 Members